DESIGN AND ANALYSIS OF ALGORITHMS

UNIT-2 NOTES

Divide-and-Conquer Method: General method

Control Abstraction

Binary search

Finding maximum and minimum

Merge sort

Quick sort

Brute force:

Knapsack problem

Travelling salesperson problem,

Convex Hull problem

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Divide and Conquer - General Method

Given a function to compute on 'n' inputs the 'divide-and-conquer' strategy suggests splitting the inputs into 'k' distinct subsets, $1 < k \leq n$, yielding 'k' subproblems. These subproblems must be solved, and then a method must be found to combine subsolutions into a solution of the whole. If the sub-problems are still relatively large, then the divide-and-conquer strategy can be reapplied. Often the subproblems resulting from a divide-and-conquer design are of the same type as the original problem. Now smaller and smaller subproblems of the same kind are generated until eventually subproblems that are small enough to be solved without splitting are produced.

The divide-and-conquer strategy splits the inpnt into subproblems of the same kind as the original problem.

UMRANA SIDDIQUI
Asst. Professor
CSE,

Control abstraction for divide and conquer

```
1 Algorithm DAndC(P)
2 {
3   if Small(P) then return S(P);
4   else
5    {
6      divide P into smaller instances P1, P2, ...., Pk, k ≥ 1;
7      Apply DAndC to each of these subproblems;
8      return Combine(DAndC(P1), DAndC(P2), ...., DAndC(Pk));
9    }
10 }
```

DAndC is initially invoked as DAndC(P), where P is the problem to be solved.

Small(P) is a boolean-valued function that determines whether the input size is small enough that the answer can be computed without splitting. If this is so, the function S is invoked. Otherwise the problem P is divided into smaller subproblems. These subproblems $P_1, P_2, \ldots, P_k$ are solved by recursive applications of DAndC.

Combine is a function that determines the solution to P using the solutions to the k subproblems. If the size of P is n and the sizes of the k subproblems are $n_1, n_2, \ldots, n_k$, respectively, then the computing time of DAndC is

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

described by the recurrence relation

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1)+T(n_2)+\ldots+T(n_k)+f(n) & \text{otherwise} \end{cases}$$

where $T(n)$ is the time for DAndC on any input of size 'n' and $g(n)$ is the time to compute the answer directly for small inputs. The function $f(n)$ is the time for dividing P and combining the solutions to subproblems.

The complexity of many divide-and-conquer algorithms is given by recurrences of the form

$$T(n) = \begin{cases} T(1) & n=1 \\ aT(n/b)+f(n) & n>1 \end{cases}$$

where 'a' and 'b' are known constants. Assuming $T(1)$ is known and 'n' is a power of b i.e. $n = b^k$.

Methods for Solving Recurrence Relations :

The recurrence relations can be solved by the following methods :

1. Substitution Method
2. Master Method

SUMRANA SIDDIQUI
Asst. Professor
CSE

Substitution Method

This method repeatedly makes substitution for each occurrence of the function T in the right-hand side until all such occurrences disappear.

Example 1:

Solve the following recurrence relation :

$$T(n) = \begin{cases} 2 & \text{if } n=1 \\ 2T(n/2)+n & \text{if } n \geq 1 \end{cases}$$

Solution

$T(n) = 2 \cdot T(n/2) + n$ Here $a=2$ $b=2$ $f(n)=n$

$= 2 \cdot [2T(n/4) + n/2] + n$ [Putting $n = n/2$ in $T(n)$]

$= 4T(n/4) + 2n$

$= 4[2T(n/8) + n/4] + 2n$

$= 8T(n/8) + 3n$

$\vdots$

$= 2^k T(n/2^k) + kn$

$= n \cdot T(1) + \log_2^n \cdot n$ [Putting $n = 2^k \Rightarrow k = \log_2^n$]

$= n \log_2^n + 2n$ [$\because T(1) = 2$]

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Example 2 :

Solve the following recurrence relation :

$$T(n) = \begin{cases} k & n = 1 \\ 3T(n/2) + kn & n > 1, n \text{ is a power of } 2 \end{cases}$$

Solution

$$T(n) = 3T(n/2) + kn$$

$$= 3\left[3T(n/4) + k(n/2)\right] + kn$$

$$= 3^2 T(n/4) + 3kn/2 + kn$$

$$= 3^2\left[3T(n/8) + \frac{kn}{4}\right] + \frac{3kn}{4} + kn$$

$$= 3^3 T(n/2^3) + \left(\frac{3}{2}\right)^2 kn + \left(\frac{3}{2}\right) kn + kn$$

$$= 3^3 T(n/2^3) + kn\left[1 + \left(\frac{3}{2}\right) + \left(\frac{3}{2}\right)^2\right]$$

$$\vdots$$

$$= 3^k T(n/2^k) + kn\left[1 + \left(\frac{3}{2}\right) + \left(\frac{3}{2}\right)^2 + \ldots + \left(\frac{3}{2}\right)^{k-1}\right]$$

$$= 3^k T(n/2^k) + \frac{kn\left(\left(\frac{3}{2}\right)^k - 1\right)}{\frac{3}{2} - 1} \qquad \left[\because 1 + x + x^2 + \ldots x^{n-1} = \frac{x^n - 1}{x - 1}\right]$$

$$= 3^k T(n/2^k) + 2kn\left(\left(\frac{3}{2}\right)^k - 1\right)$$

Put $n = 2^k \Rightarrow k = \log_2 n$

$$= 3^{\log_2 n} T(1) + 2kn\left(\left(\frac{3}{2}\right)^{\log_2 n} - 1\right)$$

SUMRANA SIDDIQUI
Asst. Professor
CSE,

$$= k \cdot 3^{\log_2 n} + 2kn \frac{3^{\log_2 n}}{2^{\log_2 n}} - 2kn \qquad [\because T(1) = k]$$

$$= k 3^{\log_2 n} + 2k\left(2^{\log_2 n}\right) \frac{3^{\log_2 n}}{2^{\log_2 n}} - 2kn \qquad \left[\begin{array}{l} n = 2^k \Rightarrow k = \log_2 n \\ = 2^{\log_2 n} \end{array}\right.$$

$$= 3^{\log_2 n} \left[k + 2k\left(\frac{2^{\log_2 n}}{2^{\log_2 n}}\right)\right] - 2kn$$

$$= 3^{\log_2 n} \left[k + 2k\right] - 2kn$$

$$= 3k \cdot 3^{\log_2 n} - 2kn$$

Example 3 :

Solve the following recurrence relation

$T(n) = 2T(n/2) + 2$, given $T(1) = 0$, $T(2) = 1$

Solution

$$T(n) = 2T(n/2) + 2$$

$$= 2\left[2T(n/4) + 2\right] + 2$$

$$= 2^2 T(n/4) + 2^2 + 2$$

$$= 2^2\left[2T(n/8) + 2\right] + 2^2 + 2$$

$$= 2^3 T(n/2^3) + 2^3 + 2^2 + 2$$

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

$$= 2^{k-1} T\left(\frac{n}{2^{k-1}}\right) + 2^{k-1} + \ldots + 2^2 + 2 + 1 - 1$$

$$= \frac{2^k}{2} T\left(\frac{n \cdot 2}{2^k}\right) + \frac{2^k - 1}{2-1} - 1 \quad \left[\because 1 + x + x^2 + x^3 + \ldots x^{n-1} = \frac{x^n - 1}{x-1}\right]$$

$$= \frac{2^k}{2} T\left(\frac{2n}{2^k}\right) + 2^k - 2$$

$$= \frac{n}{2} T(2) + n - 2 \qquad [\because n = 2^k]$$

$$= \frac{n}{2} \cdot 1 + n - 2 \qquad [\because T(2) = 1]$$

$$= \frac{3n}{2} - 2$$

Master Method

This method provides a standard method for solving recurrences of the form

$$T(n) = a\,T(n/b) + f(n) \quad \text{for } n = b^k \text{ where } k = 1, 2, 3, \ldots$$

$$T(1) = c$$

where $a \geq 1$, $b > 1$ and $c > 0$ are constants. $f(n)$ is an asymptotically positive function. This recurrence relation describes the running time of an algorithm that divides a given problem of size n into subproblems with each

SUMRANA SIDDIQUI
Asst. Professor
CSE,

one having a size n/b. Now, each subproblems is solved recursively in time $T(n/b)$. The function $f(n)$ gives the cost of dividing and combining.

If $f(n)$ is of the order of $\theta(n^d)$ where $d \geq 0$ then, by using the following relation time complexity can be calculated

$$T(n) = \begin{cases} \theta(n^d) & \text{if } a < b^d \\ \theta(n^d \log n) & \text{if } a = b^d \\ \theta(n^{\log_b^a}) & \text{if } a > b^d \end{cases}$$

Example 1 :

Consider the given recurrence relation, $T(n) = 9T(n/3) + n$

Solution

$a = 9 \quad b = 3 \quad d = 1 \quad f(n) = n$

$a > b^d \Rightarrow 9 > 3 \quad \therefore T(n) = \theta(n^{\log_b^a})$

$$\begin{aligned} T(n) &= \theta(n^{\log_b^a}) \\ &= \theta(n^{\log_3^9}) \\ &= \theta(n^{\log_3^{3^2}}) \\ &= \theta(n^{2\log_3^3}) \\ &= \theta(n^{2 \cdot 1}) \quad [\because \log_n^n = 1] \\ &= \underline{\theta(n^2)} \end{aligned}$$

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Example 2 :

Consider the given recurrence relation, $T(n) = 2T(n/2) + 1$

Solution

$a = 2 \quad b = 2 \quad d = 0 \quad f(n) = 1$

$a > b^d \Rightarrow 2 > 2^0 \quad \therefore T(n) = \theta(n^{\log_b^a})$

$T(n) = \theta(n^{\log_b^a})$

$= \theta(n^{\log_2^2})$

$= \theta(n)$

Example 3 :

Consider the given recurrence relation, $T(n) = 9T(n/3) + 4n^6$

Solution

$a = 9 \quad b = 3 \quad d = 6 \quad f(n) = 4n^6$

$a < b^d \Rightarrow 9 < 3^6 \quad \therefore T(n) = \theta(n^d)$

$T(n) = \theta(n^d)$

$= \theta(n^6)$

$= \theta(n^6)$

SUMRANA SIDDIQUI
Asst. Professor
CSE

Binary Search

Let a_i, $1 \le i \le n$, be a list of elements that are sorted in nondecreasing order.
Consider the problem of determining whether a given element 'x' is present in the list.

- If 'x' is present, determine a value 'j' such that $a_j = x$.
- If 'x' is not in the list, then 'j' is set to be zero.

Let $P = (n, a_i, \ldots a_l, x)$ denote an arbitary instance of this problem where n is number of elements in the list $a_i, \ldots a_l$, and x is the element searched for.

Divide-and-conquer can be used to solve this problem.
Let Small(P) be true if $n = 1$. In this case, S(P) will take the value i if $x = a_i$; otherwise it will take the value 0.
If P has more than one element, it can be divided into a new subproblem by picking an index 'q' in the range $[i, l]$ such that $q = \frac{i+l}{2}$ and comparing 'x' with 'a_q'. There are three possibilities:

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

(1) $x = a_q$: problem P is immediately solved.

(2) $x < a_q$: x has to be searched for only in the sublist $a_i, a_{i+1}, \ldots a_{q-1}$. Therefore P reduces to $(q-i, a_i, \ldots a_{q-1}, x)$

(3) $x > a_q$: x has to be searched for only in the sublist $a_{q+1}, \ldots, a_l$. P reduces to $(l-q, a_{q+1}, \ldots, a_l, x)$.

In this, any given problem P gets divided into one new subproblem. After a comparision with a_q, the instance remaining to be solved (if any) can be solved by using this divide-and-conquer scheme again. If 'q' is always chosen such that a_q is the middle element, then the resulting search algorithm is known as binary search.

Recursive binary search

```
1  Algorithm BinSrch(a, i, l, x)
2  // Given an array a[i:l] of elements in nondecreasing
3  // order, 1 ≤ i ≤ l, determine whether x is present, and
4  // if so, return j such that x = a[j]; else return 0.
5  {
6     if (l = i) then // If Small(P)
7     {
8        if (x = a[i]) then return i;
9        else return 0;
10    }
```

```
11    else
12    {// Reduce P into a smaller subproblem.
13       mid := ⌊(i + l)/2⌋;
14       if (x = a[mid]) then return mid;
15       else if (x < a[mid]) then
16            return BinSrch(a, i, mid-1, x);
17       else return BinSrch(a, mid+1, l, x);
18    }
19 }
```

Iterative binary search

```
1  Algorithm BinSearch(a, n, x)
2  // Given an array [1:n] of elements in nondecreasing
3  // order, n ≥ 0, determine whether x is present, and
4  // if so, return j such that x = a[j]; else return 0.
5  {
6     low := 1; high := n;
7     while (low ≤ high) do
8     {
9        mid := ⌊(low + high)/2⌋;
10       if (x < a[mid]) then high := mid - 1;
11       else if (x > a[mid]) then low := mid + 1;
12          else return mid;
13    }
14    return 0;
15 }
```

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Time complexity : The computing time of binary search is given by formulas that describe the best, average, and worst cases:

Successful searches	Unsuccessful searches
$\theta(1), \theta(\log n), \theta(\log n)$	$\theta(\log n)$
best, average, worst	best, average, worst

→ Variation of Binary Search : In this algorithm only one comparision per iteration of the while loop takes place.

```
1 Algorithm BinSearch1 (a, n, x)
2 // Same specifications as BinSearch except n>0.
3 {
4    low := 1; high := n+1;
5    // high is one more than possible.
6    while (low < (high - 1)) do
7    {
8       mid := ⌊(low+high)/2⌋;
9       if (x < a[mid]) then high := mid;
10         // Only one comparision in the loop.
11      else low := mid; // x ≥ a[mid]
12   }
13   if (x = a[low]) then return low; // x is present.
14   else return 0; // x is not present.
15 }
```

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET.

Example: Consider an array consisting of elements 1, 2, 5, 6, 7, 10, 24, 56, 84, 100, 115, 120.

① Let $x = 120$

$n = 12$

low	High	mid	A[mid]	X ? A[mid]
1	12	6	10	120 > 10
7	12	9	84	120 > 84
10	12	11	115	120 > 115
12	12	12	120	found

② Let $x = 9$

$n = 12$

low	high	mid	A[mid]	X ? A[mid]
1	12	6	10	9 < 10
1	5	3	5	9 > 5
4	5	4	6	9 > 6
5	5	5	7	9 > 7
6	5	-	-	Not found

low > high Element not found.

Merge Sort

Another example of divide and conquer, is a sorting algorithm with its complexity as $O(n \log n)$ in the worst case. This algorithm is called 'merge sort'. The elements in this get sorted in non decreasing order.

Given a sequence of 'n' elements $a[1], \ldots, a[n]$, the general idea is to split them into two sets $a[1], \ldots, a[n/2]$ and $a[\lfloor n/2 \rfloor + 1], \ldots, a[n]$. Each set is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of 'n' elements. This is another ideal example of the divide-and-conquer strategy in which the splitting is into two equal-sized sets and the combining operation is the merging of two sorted sets into one.

'MergeSort' Algorithm describes this process using recursion and a function 'Merge' merges the two sorted sets. Before executing 'MergeSort', the 'n' elements should be placed in $a[1:n]$.

The MergeSort $(1, n)$ causes the elements to be rearranged into non decreasing order in a.

SUMRANA SIDDIQUI
Asst. Professor

Merge Sort

```
1 Algorithm MergeSort (low, high)
2 //a[low:high] is a global array to be sorted.
3 // Small(P) is true if there is only one element.
4 // to sort. In this case the list is already sorted.
5 {
6    if (low < high) then // If more than one element.
7    {
8       // Divide P into subproblems.
9          // Find where to split the set.
10            mid := ⌊(low + high)/2⌋;
11         // Solve the subproblems.
12           MergeSort (low, mid);
13           MergeSort (mid+1, high);
14         // Combine the solutions.
15           Merge (low, mid, high);
16   }
17 }
```

Merging two sorted arrays using auxiliary storage

```
1 Algorithm Merge (low, mid, high)
2 // a[low:high] is a global array containing two sorted
3 // subsets in a[low:mid] and in a[mid+1, high]. The goal
4 // is to merge these two sets into a single set residing
5 // in a[low:high]. b[] is an auxiliary global array.
6 {
7    h := low; i := low; j := mid + 1;
```

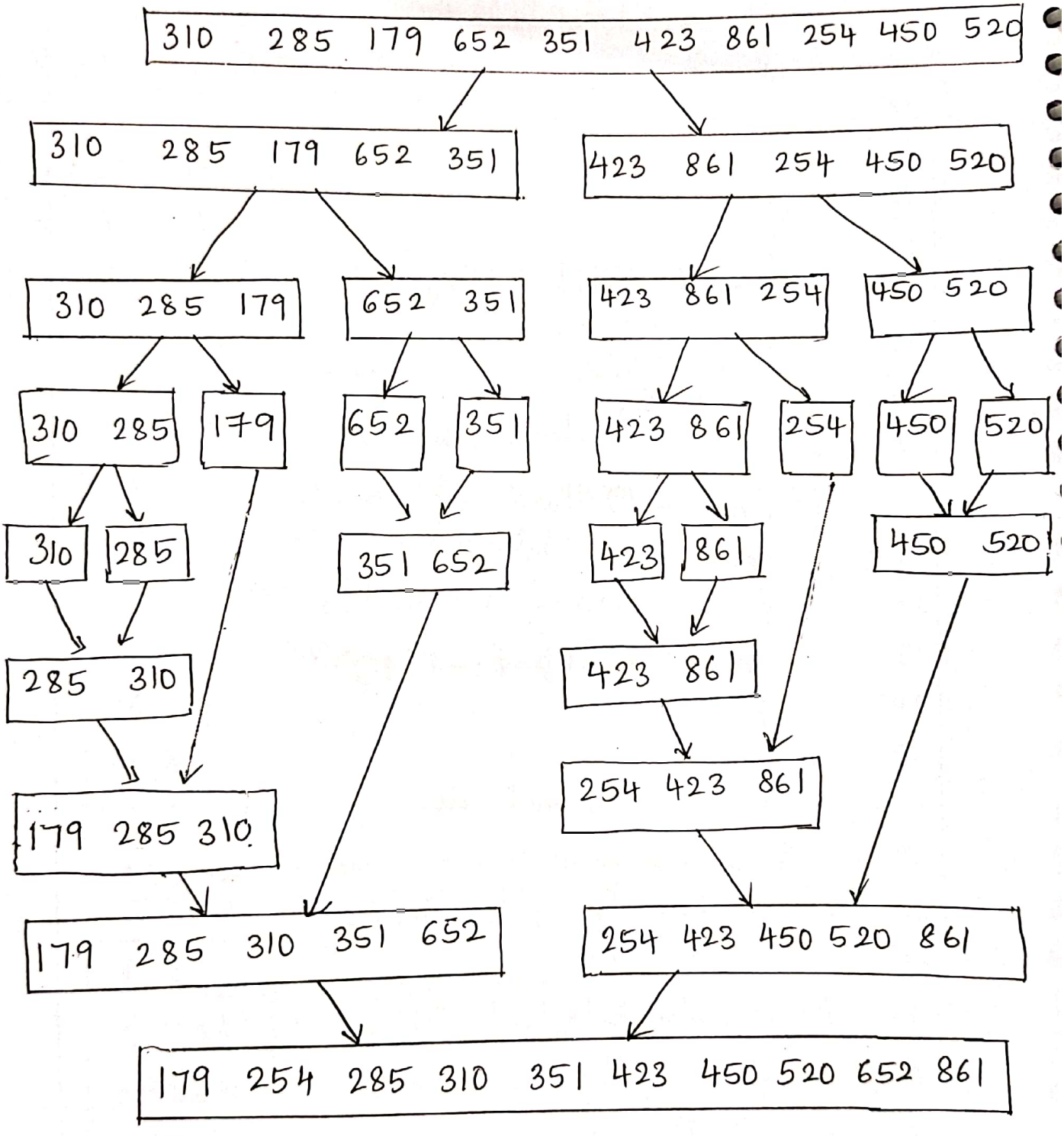
SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

```
8    while ((h ≤ mid) and (j ≤ high)) do
9    {
10       if (a[h] ≤ a[j]) then
11       {
12          b[i] := a[h]; h := h+1;
13       }
14       else
15       {
16          b[i] := a[j]; j := j+1;
17       }
18       i := i+1;
19   }
20   if (h > mid) then
21       for k := j to high do
22       {
23          b[i] := a[k]; i := i+1;
24       }
25   else
26       for k := h to mid do
27       {
28          b[i] := a[k]; i := i+1;
29       }
30   for k := low to high do a[k] := b[k];
31 }
```

SUMRANA SIDDIQUI
Asst. Professor

Example: Consider the array of ten elements a[1:10] = (310, 285, 179, 652, 351, 423, 861, 254, 450, 520).

Solution



	1	2	3	4	5	6	7	8	9	10
A	179	254	285	310	351	423	450	520	652	861

A[1:10]

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

If the time for the merging operation is proportional to n, then the computing time for merge sort is described by the recurrence relation

$$T(n) = \begin{cases} a & n=1, \text{'a' a constant} \\ 2T(n/2)+cn & n>1, \text{'c' a constant} \end{cases}$$

When n is a power of 2, $n = 2^k$

$$\begin{aligned} T(n) &= 2\left[2T(n/4)+cn/2\right]+cn \\ &= 2^2\left[T(n/4)+2cn\right. \\ &= 2^2\left[2T(n/8)+cn/4\right]+2cn \\ &= 2^3\left[T(n/2^3)\right]+3cn \\ &\vdots \\ &= 2^k\left[T(n/2^k)\right]+kcn \\ &= 2^k T(1)+kcn \qquad [\because n=2^k] \\ &= an+kcn \qquad [\because k=\log_2 n] \\ &= an+cn\log n \end{aligned}$$

if $2^k < n \le 2^{k+1}$, then $T(n) \le T(2^{k+1})$

Therefore $T(n) = O(n\log n)$

→ The space complexity of 'Merge Sort' is $O(\log n)$.

Quick Sort

The divide-and-conquer approach can be used for another efficient sorting method different from merge sort.

In 'merge sort', the file a[1:n] is divided at its midpoint into subarrays which were independently sorted and later merged.

In 'quick sort', the division into two subarrays is made so that the sorted subarrays do not need to be merged later. This is accomplished by rearranging the elements in a[1:n] such that $a[i] \leq a[j]$ for all 'i' between 1 and 'm' and all 'j' between 'm+1' and 'n' for some m, $1 \leq m \leq n$. Thus, the elements in a[1:m] and a[m+1:n] can be independantly sorted. No merge is needed. The rearrangement of the elements is accomplished by picking some element of a[], say t = a[s], and then reordering the other elements so that all elements appearing before t in a[1:n] are less than or equal to 't' and all elements appearing after 't' are greater than or equal to 't'. This rearranging is referred to as 'partitioning'.

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

QuickSort

```
1  Algorithm QuickSort (p,q)
2  // Sorts the elements a[p],...,a[q] which reside in the
3  // global array a[1:n] into ascending ordu; a[n+1]
4  // is defined and must be ≥ all elements in a[1:n].
5  {
6      if (p<q) then // If there are more than one element
7      {
8          // divide P into two subproblems.
9              j := Partition (a, p, q+1);
10                 // j is the position of the partitioning element.
11         // Solve the subproblems.
12             QuickSort (p, j-1);
13             QuickSort (j+1, q);
14         // There is no need for combining solutions.
15     }
16 }
```

```
1  Algorithm Partition (a, m, p)
2  // Within a[m], a[m+1],..,a[p-1] the elements are
3  // rearranged in such a manner that if initially t=a[m],
4  // then after completion a[q]=t for some q between m
5  // and p-1, a[k]≤t for m≤k<q, and a[k]≥t
6  // for q<k<p, q is returned. Set a[p]=∞.
7  {
8      v := a[m]; i := m; j := p;
9      repeat
```

SUMRANA SIDDIQUI
Asst. Professor
CSE.

```
10      {
11        repeat
12          i := i+1;
13        until (a[i] ≥ v);
14        repeat
15          j := j+1;
16        until (a[j] ≤ v);
17        if (i<j) then Interchange (a,i,j);
18      } until (i ≥ j);
19      a[m] := a[j]; a[j] := v; return j;
20  }
```

```
1 Algorithm Interchange (a,i,j)
2 // Exchange a[i] with a[j].
3 {
4       p := a[i];
5       a[i] := a[j]; a[j] := p;
6 }
```

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Example : Perform Quick Sort technique on the following array of elements :

{ 65, 70, 75, 80, 85, 60, 55, 50, 45. }

Solution :

(65) pivot	70 i↑	75	80	85	60	55	50	45 j↑

rule:

> if (i < pivot) → increment i
> (j > pivot) → decrement j
> else swap both i and j if $i < j$
> if $i \geq j$ then swap pivot with j

(65)	70 i	75	80 swap i & j	85	60	55	50	45 j
(65)	45 i	75	80	85	60	55	50	70 j
(65)	45	75 i	80	85 swap i,j	60	55	50 j	70
(65)	45	50 i	80	85	60	55	75 j	70
(65)	45	50	80 i	85 swap i,j	60	55 j	75	70
(65)	45	50	55 i	85	60	80 j	75	70
(65)	45	50	55	85 i	60 j	80	75	70
(65)	45	50	55	60 i	85 j	80	75	70
(65)	45	50	55	60 j	85 i	80	75	70 i > j swap pivot, j

pivot

60 45 50 55 | 65 | 85 80 75 70

subarray 1 | subarray 2

(60) 45 50 55 | 65 | (85) 80 75 70
i j i j

(60) 45 50 55 | 65 | 85 80 75 70
i j i j

(60) 45 50 55 | 65 | 85 80 75 70
ij i j

(60) 45 50 55 | 65 | (85) 80 75 70
ji ij

i=j swap pivot & j

55 45 50 | 60 | 65 | 70 | 80 75 85

(55) 45 50 | 60 | 65 | 70 | (80) 75 85
i j i j

(55) 45 50 | 60 | 65 | 70 | (80) 75 85
ij i j

swap pivot & j | i swap pivot & j

50 45 | 55 | 60 | 65 | 70 | 75 | 80 | 85

(50) 45 | 55 | 60 | 65 | 70 | 75 | 80 | 85
ij

swap pivot & j

45 | 50 | 55 | 60 | 65 | 70 | 75 | 80 | 85

∴ 45, 50, 55, 60, 65, 70, 75, 80, 85

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

→ Time Complexity of Quick Sort

① **Best case** : It works best if each array is divided into two equal subarrays of size $n/2$.

$$T(n) = O(n \log n)$$

② **Worst case** : In worst case, the chosen point is either the smallest or the largest element is an array. In this case, one part is empty and the other part contains the remaining elements.

$$T(n) = O(n^2)$$

③ **Average case** : In average case, the array is partitioned by choosing any random number. In this case at each level some of the partitions are well balanced while some are fairly unbalanced.

$$T(n) = O(n \log n)$$

→ The Space complexity of Quick Sort is $O(\log n)$.

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Derivation of the average case time complexity of Quick Sort

In the average case, the array is partitioned by choosing any random number. In this case, at each level some of the partitions are well-balanced while some are fairly unbalanced.

The time complexity for Quick Sort is described by the recurrence relation,

$$T(n) = \begin{cases} a & n=1 \text{ , 'a' is a constant} \\ 2T(n/2)+cn & n>1 \text{ , 'c' is a constant} \end{cases}$$

when n is a power of 2, $n = 2^k$

$$T(n) = 2\left[2T(n/4) + cn/2\right] + cn$$

$$= 2^2\left[T(n/4)\right] + 2cn$$

$$= 2^2\left[2T(n/8) + cn/4\right] + 2cn$$

$$= 2^3\left[T(n/2^3)\right] + 3cn$$

$$\vdots$$

$$= 2^k\left[T(n/2^k)\right] + kcn$$

$$= 2^k T(1) + kcn$$

$$= an + kcn \qquad [\because k = \log_2 n]$$

$$= an + cn\log n$$

if $2^k < n \le 2^{k+1}$, then $T(n) \le T(2^{k+1})$

Therefore, $\boxed{T(n) = O(n\log n)}$

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Finding the Maximum and Minimum

The problem to find the maximum and minimum items in a set of "n" elements can be solved by the divide-and-conquer technique.

A straightforward conventional algorithm for finding the maximum and minimum items is as follows:

```
1 Algorithm StraightMaxMin (a, n, max, min)
2 // Set max to the maximum and min to the minimum
3 // of a[1:n]
4 {
5     max := min := a[1];
6     for i := 2 to n do
7     {
8         if (a[i] > max) then max := a[i];
9         if (a[i] < min) then min := a[i];
10    }
11 }
```

The best case occurs when the elements are in increasing order and thus the number of element comparisions is $n-1$.

The worst case occurs when the elements are in decreasing order and thus the number of element comparisions is $2(n-1)$.

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

In the average case the number of comparisions would be $\frac{n-1 + 2(n-1)}{2} = \frac{3n-2}{2}$.

→ A divide-and-conquer approach for this problem would proceed as follows:

Let $P = (n, a[i], \ldots, a[j])$ denote an arbitary instance. Here n is the number of elements in the list $a[i], \ldots, a[j]$ from where the maximum and minimum element is to be found. Let Small(P) be true when $n \leq 2$.

- the maximum and minimum are $a[i]$ if $n = 1$.
- if $n = 2$, the problem can be solved by making one comparision.
- if $n > 2$, the list has more than two elements, P has to be divided into smaller instances.

$P_1 = (\lfloor n/2 \rfloor, a[1], \ldots a[\lfloor n/2 \rfloor])$ and

$P_2 = (n - \lfloor n/2 \rfloor, a[\lfloor n/2 \rfloor + 1], \ldots, a[n])$.

After having divided P into two smaller subproblems, they can be solved by recursively invoking the same divide-and-conquer algorithm. The solutions for P_1 and P_2 can be combined to obtain the solution of P. If MAX(P) and MIN(P) are the maximum and minimum

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

of the elements in P, then MAX(P) is the larger of MAX(P_1) and MAX(P_2). Also, MIN(P) is the smaller of MIN(P_1) and MIN(P_2).

```
1 Algorithm MaxMin(i, j, max, min)
2 // a[1:n] is a global array. Parameters i and j are integers
3 // 1 ≤ i ≤ j ≤ n. The effect is to set max and min to
4 // the largest and smallest values in a[i:j], respectively.
5 {
6   if (i = j) then max := min := a[i]; // Small(P)
7   else if (i = j-1) then // Another case of Small(P)
8     {
9       if (a[i] < a[j]) then
10      {
11        max := a[j]; min := a[i];
12      }
13      else
14      {
15        max := a[i]; min := a[j];
16      }
17    }
18    else
19    { // If P is not small, divide P into subproblems.
20      // Find where to split the set.
21        mid := ⌊(i+j)/2⌋;
22      // Solve the subproblems.
23        MaxMin(i, mid, max, min);
24        MaxMin(mid+1, j, max1, min1);
25      // Combine the solutions.
26        if (max < max1) then max := max1;
27        if (min > min1) then min := min1;
28    }
29 }
```

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

In the analysis of algorithm, $T(n)$ represents the number of element comparisions needed, then the resulting recurrence relation is

$$T(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ T(\lceil n/2 \rceil) + T(\lceil n/2 \rceil) + 2 & n > 2 \end{cases}$$

When n is a power of two, $n = 2^k$ for some positive integer k, then

$$\begin{aligned} T(n) &= T(n/2) + T(n/2) + 2 \\ &= 2T(n/2) + 2 \\ &= 2(2T(n/4) + 2) + 2 \\ &= 2^2 T(n/2^2) + 2^2 + 2 \\ &= 2^2(2T(n/8) + 2) + 2^2 + 2^1 \\ &= 2^3 T(n/2^3) + 2^3 + 2^2 + 2^1 \\ &= 2^{k-1} T(n/2^{k-1}) + 2^{k-1} + 2^{k-2} + \cdots 2^1 \\ &= 2^{k-1} T\left(\frac{2^k}{2^{k-1}}\right) + 2^{k-1} + 2^{k-2} + \cdots 2^1 \quad [\because n = 2^k] \\ &= 2^{k-1} T(2) + (2^{k-1} + 2^{k-2} + \cdots 2) \\ &= 2^{k-1} T(2) + \frac{2^{k+1} - 1}{2 - 1} \end{aligned}$$

$$\left[\because x^{n-1} + x^{n-2} + \cdots 1 = \frac{x^{n+1} - 1}{(x-1)}\right]$$

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

$$2^{k-1} T(2) + 2^k - 1 \quad [\because T(2) = 1]$$

$$\frac{2^k}{2} + 2^k - 1 \quad [\because 2^k = n]$$

$$\frac{n}{2} + n - 1$$

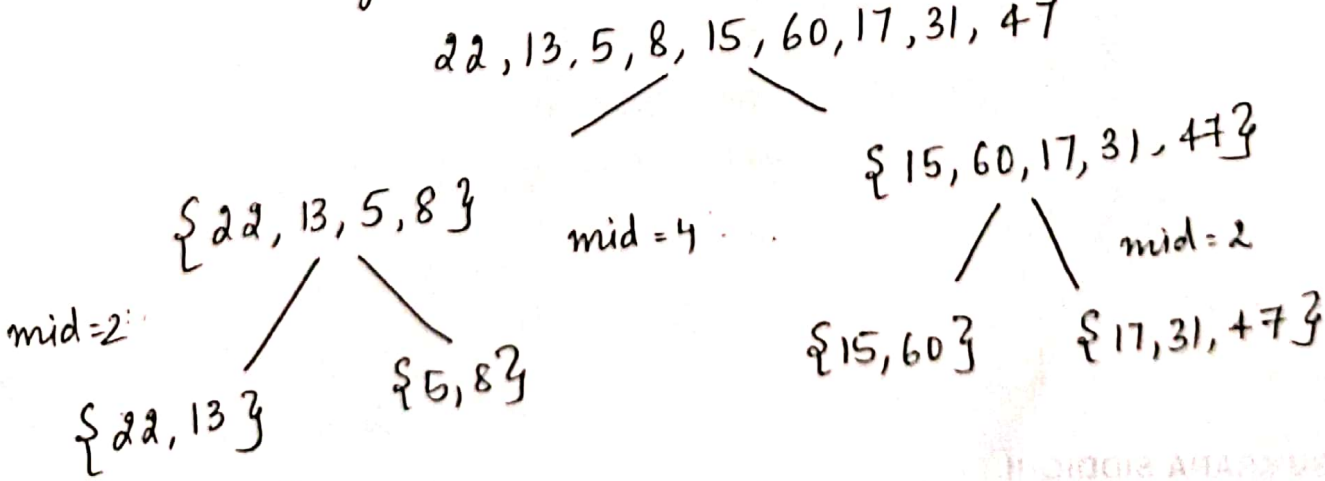
$$\frac{n + 2n - 2}{2} = \frac{3n-2}{2} = \underline{\underline{\frac{3n-2}{2}}}.$$

Hence $\frac{3n}{2} - 2$ is the best, average and worst case number of comparisions when n is a power of two.

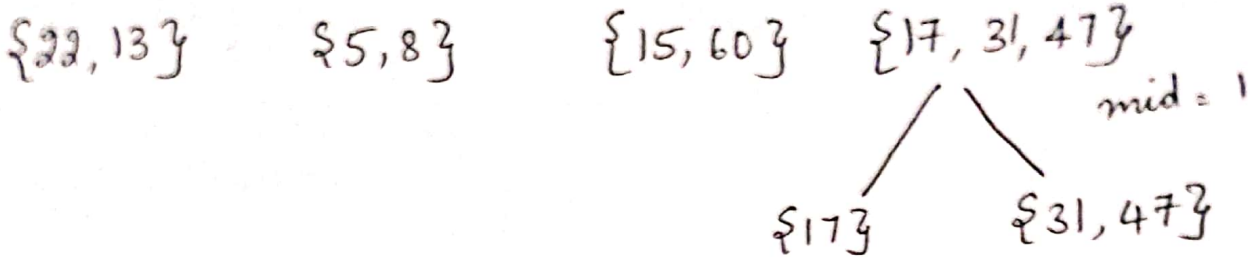
Example: Find the maximum and minimum element from the following list of elements using Divide and Conquer method.
The list is {22, 13, 5, 8, 15, 60, 17, 31, 47}.

Solution: 22, 13, 5, 8, 15, 60, 17, 31, 47

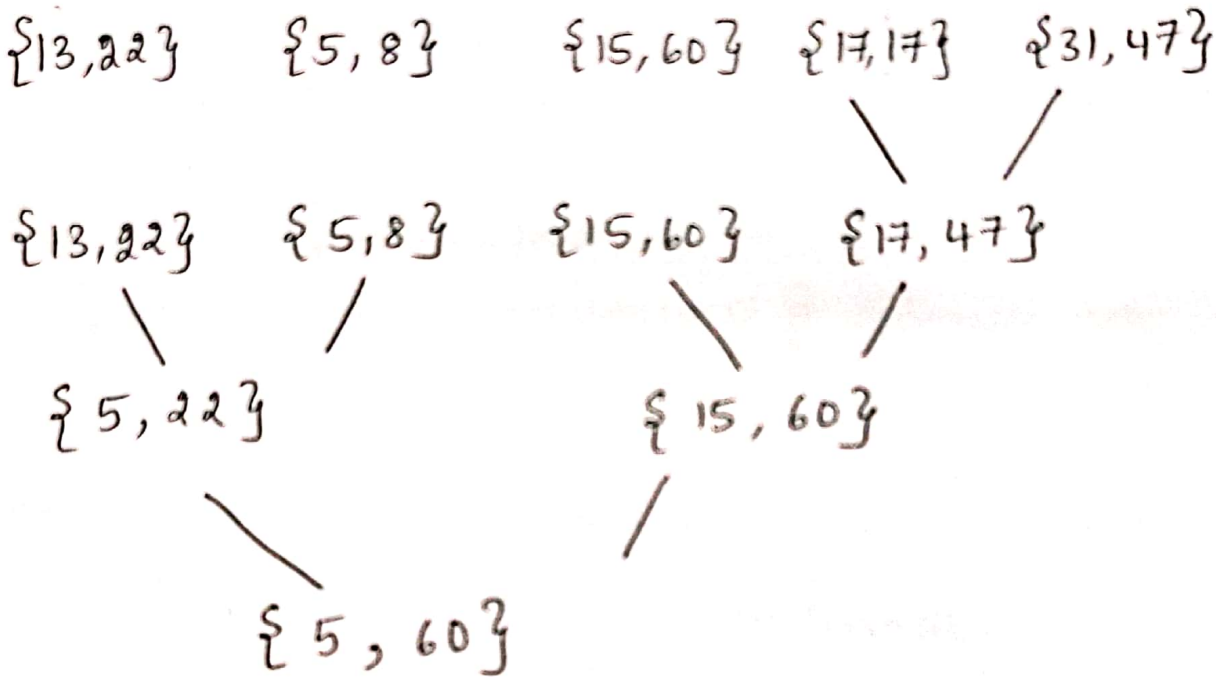
Dividing it into subproblems recursively until small.



SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET



Solve subproblems and combine



The maximum and minimum elements of the problem are 60 and 5.

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Brute Force - General Method

'Brute force' is a straightforward approach to solving a problem, usually directly based on the problem's statement and definitions of the concepts involved.

The "force" implied by the strategy's definition is that of a computer.

Example - compute a^n for a given number 'a' and a nonnegative integer n. By the definition of exponentiation,

$$a^n = \underbrace{a * \ldots * a}_{n \text{ times}}.$$

This suggests computing a^n by multiplying 'a' 'n' times.

Brute-force strategy is applicable to a very wide variety of problems. It is used for many elementary but important algorithmic tasks such as computing the sum of 'n' numbers, finding the largest element in the list. For some problems such as sorting, searching, matrix multiplication, string matching, the brute-force approach yields resonable algorithms of at least some practical

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

value with no limitation on instance size.

The expense of designing a more efficient algorithm may be unjustifiable if only a few instances of a problem need to be solved and a brute-force algorithm can solve those instances with acceptable speed.

Even if too inefficient in general, a brute-force algorithm can still be useful for solving small-size instances of a problem.

Finally, a brute-force algorithm can serve an important theoretical or educational purpose, as a yardstick with which to judge more efficient alternatives for solving a problem.

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Brute - Force String Matching

In the string-matching problem : a given string of 'n' characters called the 'text' and a string of 'm' characters ($m \leq n$) called the 'pattern', find a substring of the text that matches the pattern.

A 'brute-force algorithm' for the string-matching problem is : align the pattern against the first 'm' characters of the text and start matching the corresponding pairs of characters from left to right until either all the 'm' pairs of the characters match or a mismatching pair is encountered. In the latter case, the pattern is shifted one position to the right and character comparisions are resumed, starting again with the first character of the pattern and its counterpart in the text. The last position in the text that can still be a beginning of a matching substring is $n-m$, provided the texts' positions are indexed from 0 to $n-1$. Beyond that position, there are not enough characters to match the entire pattern; hence, the algorithm need not to make any comparisions there.

SUMRANA SIDDIQUI
Asst. Professor
CSE.

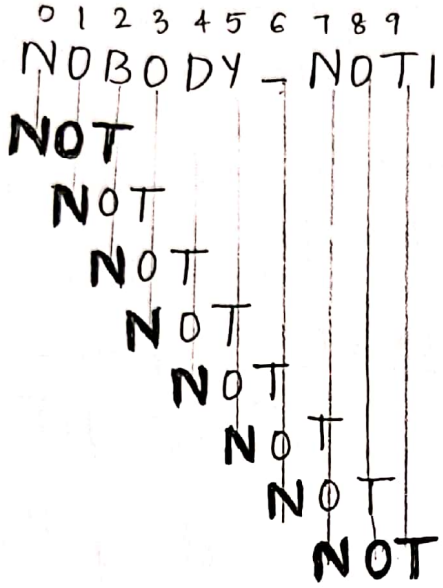
```
1 Algorithm BruteForceStringMatch(T[0..n-1], P[0..m-1])
2 //The algorithm implements brute-force string matching.
3 //Input : An array T[0..n-1] of n characters
4 //                representing a text;
5 //        An array P[0..m-1] of m characters
6 //                representing a pattern.
7 //Output : The position of the first character in the
8 //text that starts the first matching substring
9 //if the search is successful and -1 otherwise.
10 {
11     for i := 0 to n-m do
12         j := 0;
13         while((j < m) and (P[j] = T[i+j])) do
14             j := j+1;
15         if (j = m) then
16             return i;
17     return -1;
18 }
```

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Example: Find the occurences of the pattern = NOT in the text below:

NOBODY_NOTICED_HIM

Solution:



It returns index no. 7 because here the pattern matches the substring in the text.

→ The pattern's characters that are compared with their text counterparts in bold type. The algorithm shifts the pattern almost always after a single character comparision.

→ The worst case time complexity is $\underline{\underline{O(nm)}}$ and the average case complexity is $\underline{\underline{O(n)}}$.

SUMRANA SIDDIQUI
Asst. Professor
CSE

Closest-Pair Problem

The "closest-pair problem" calls for finding the two closest points in a set of 'n' points. Assuming the points are specified in a standard fashion by their (x, y) Cartesian coordinates and that the distance between two points $P_i = (x_i, y_i)$ and $P_j = (x_j, y_j)$ is the standard Euclidean distance

$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

The Brute-force approach to solving this problem leads to the following algorithm:

Compute the distance between each pair of distinct points and find a pair with the smallest distance. To avoid computing the distance between the same pair of points twice, consider only the pairs of points (P_i, P_j) for which $i < j$.

The basic operation of the algorithm is computing the Euclidean distance between two points.

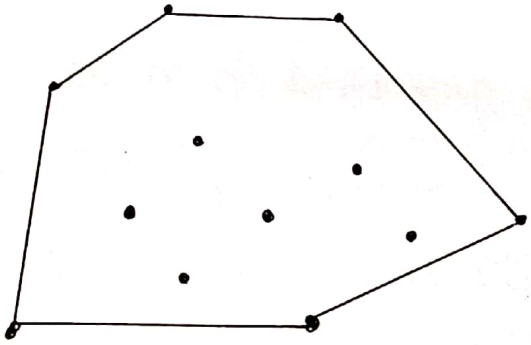
SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

```
1  Algorithm BruteForceClosestPoints(P)
2  // Input : A list P of n(n≥2) points P1=(x1,y1),
3  // ...., Pn=(xn,yn).
4  // Output : Indices index1 and index2 of the
5  // closest pair of points
6  {
7     dmin := ∞ ;
8       for i := 1 to n-1 do
9        for j := i+1 to n do
10         d := sqrt((xi-xj)^2+(yi-yj)^2)
11       if (d < dmin) then
12         dmin := d ; index1 := i ; index2 := j ;
13    return index1, index2
14 }
```

SUMRANA SIDDIQUI
Asst. Professor
CSE.

Convex-Hull Problem

A conver hull is an important structure in geometry that can be used in the construction of many other geometric structures. The convex hull of a set S of points in the plane is defined to be the smallest convex polygon containing all the points of S.

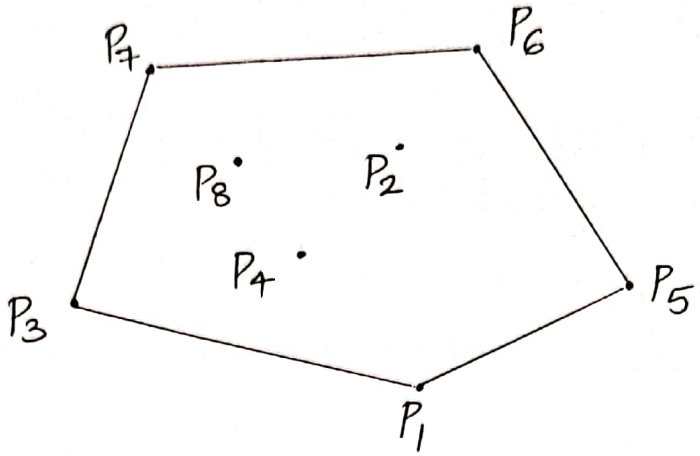


The "convex hull problem" is the problem of constructing the convex hull for a given set S of 'n' points. To solve it, find the points that will serve as the vertices of a polygon. Such vertices of a polygon are called "extreme points".

An "extreme point" of a convex set is a point of this set that is not a middle point of any line segment with end points in the set. For example, the extreme points of a triangle are its three vertices, the extreme points of a circle are all the points of its circumference, the extreme points

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

of the convex hull of the set of eights points in the fig. below are P_1, P_5, P_6, P_7 and P_3.



A simple algorithm based on the following observation about line segments making up a boundary of the convex hull : a line segment connecting two points P_i and P_j of a set of n points is a part of its convex hull's boundary if and only if all the other points of the set lie on the same side of the straight line through these two points. Repeating this test for every pair of points yields a list of line segments that make up the convex hull's boundary.

To implement this algorithm,

First, the straight line through two points (x_1, y_1) (x_2, y_2) in the coordinate plane can be defined by the equation $ax + by = c$

where $a = y_2 - y_1$

$b = x_1 - x_2$

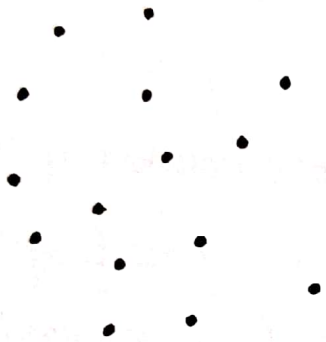
$c = x_1 y_2 - y_1 x_2$

SUMRANA SIDDIQUI
Asst. Professor
CSE.

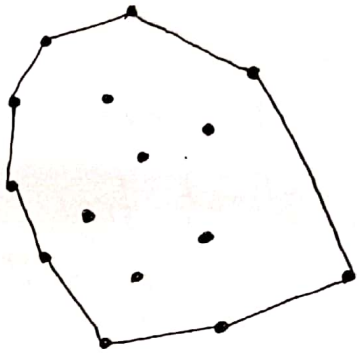
Second, such a line divides the plane into two half planes : for all the points in one of them $ax+by > c$ while for all the points in the other $ax+by < c$.

Time complexity is $O(n^3)$.

Example :



Set of points



Its convex hull

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Exhaustive Search

'Exhaustive Search' is a brute-force approach to combinatorial problems. It suggests generating each and every element of the problem's domain, selecting those of them that satisfy the problem's constraints and then finding a desired element.

Traveling Salesman Problem

The problem asks to find the shortest tour through a given set of 'n' cities that visits each city exactly once before returning to the city where it started. The problem can be modeled by a weighted graph, with the graph's vertices representing the cities and the edge weights specifying the distances. Then the problem can be stated as the problem of finding the shortest 'Hamiltonian circuit' of the graph where the Hamiltonian circuit is defined as a cycle that passes through all the vertices of the graph exactly once.

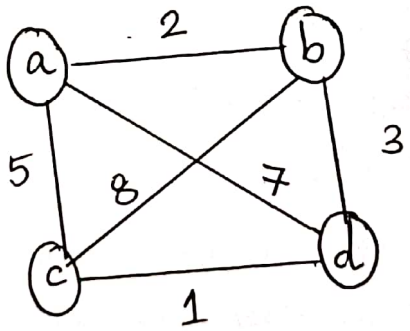
A Hamiltonian circuit can also be defined as a sequence of $n+1$ adjacent vertices $v_{i_0}, v_{i_1}, \ldots, v_{i_{n-1}}, v_{i_0}$, where the first

SUMRANA SIDDIQUI
Asst. Professor
CSE.

vertex of the sequence is the same as the last one while other n-1 vertices are distinct. Further, it can be assumed, that all circuits start and end at one particular vertex. Thus, obtain all the tours by generating all the permutations of n-1 intermediate cities, compute the tour lengths, and find the shortest among them.

The total number of permutations needed will be (n-1)!/2. This makes the exhaustive-search approach impractical for all but very small values of n.

Example :



Tour	Length
a → b → c → d → a	L = 2+8+1+7 = 18
a → b → d → c → a	L = 2+3+1+5 = 11 optimal
a → c → b → d → a	L = 5+8+3+7 = 23
a → c → d → b → a	L = 5+1+3+2 = 11 optimal
a → d → b → c → a	L = 7+3+8+5 = 23
a → d → c → b → a	L = 7+1+8+2 = 18

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

TSP Brute Force

```
1 Algorithm TSPBrute ()
2 {
3    Calculate total number of tours.
4    List all the possible tours.
5    Calculate the distance of each tour.
6    Choose the shortest tour.
7 }
```

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

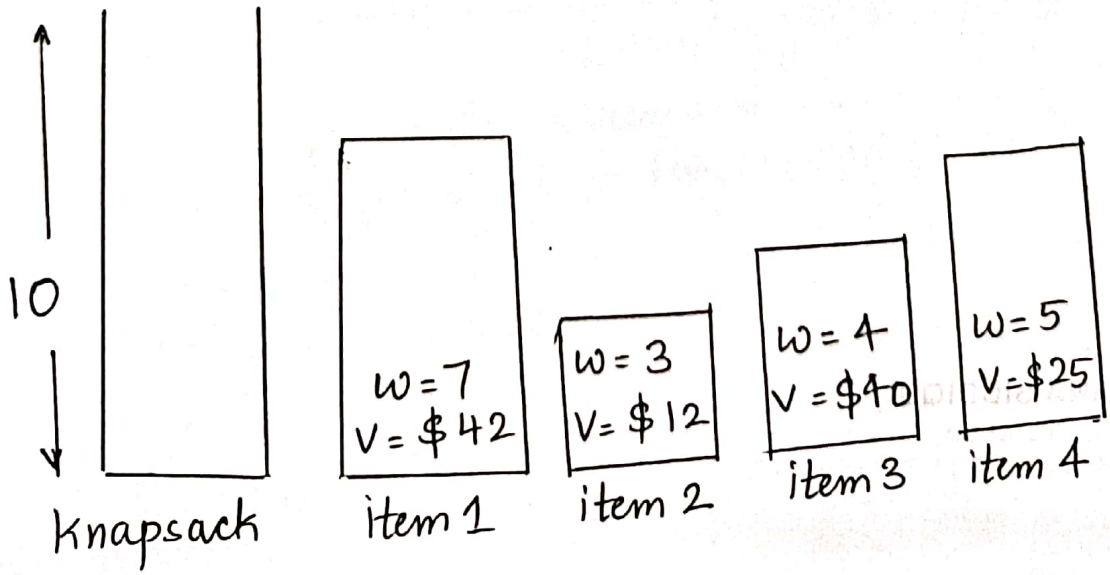
Knapsack Problem

Given 'n' items of known weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ and a knapsack of capacity W, find the most valuable subset of items that fit into knapsack.

The Exhaustive search approach to this problem leads to considering all the subsets of the set of 'n' items given, computing the total weight of each subset in order to identify feasible subsets i.e. the ones with the total weight not exceeding the knapsack's capacity and finding a subset of the largest value among them.

Since the number of subsets of an n-element set is 2^n, the exhaustive search leads to $\Omega(2^n)$ algorithm no matter how efficiently individual subsets are generated.

Example :



S. IMRANA SIDDIQUI
Asst. Professor
CSE,

Subset	Total weight	Total value	
$\emptyset$	0	$ 0	
{1}	7	$ 42	
{2}	3	$ 12	
{3}	4	$ 40	
{4}	5	$ 25	
{1,2}	10	$ 54	
{1,3}	11	not feasible	
{1,4}	12	not feasible	
{2,3}	7	$ 52	
{2,4}	8	$ 37	
{3,4}	9	$ 65	→ optimal
{1,2,3}	14	not feasible	
{1,2,4}	15	not feasible	
{1,3,4}	16	not feasible	
{2,3,4}	12	not feasible	
{1,2,3,4}	19	not feasible	

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET

Knapsack Brute Force

```
1  Algorithm BruteForce(W[], V[], A[])
2  // Finds the best possible combinations
3  // Input: Array weights contains the weights of all items
4  // Array values contains the values of all items
5  // Array A initialized with 0s
6  // Output: Best possible combinations of items in
7  // the knapsack bestChoice [1..N]
8  {
9      for i:=1 to 2^n do
10     j := n
11     tempWeight := 0; tempValue := 0;
12     while (A[j]!=0 and j>0)
13         A[j] := 0;
14         j := j-1;
15     A[j] := 1;
16     for k := 1 to n do
17         if (A[k]==1) then
18             tempWeight := tempWeight + Weights[k];
19             tempValue := tempValue + Values[k];
20 if ((tempValue > bestValue) &&
                (tempWeight ≤ Capacity) then
21
22     bestValue := tempValue;
23     bestWeight := tempWeight;
24 bestChoice := A;
25 return bestChoice;
26 }
```

SUMRANA SIDDIQUI
Asst. Professor
CSED, MJCET